



ClimatePros

Unified Architecture



A Auto Dispatch **B** Closeout Copilot **C** Parts Finder

Version 1.0 — Draft for Client Review



PREPARED FOR ClimatePros

PREPARED BY techjays

ISSUED July 28, 2026

COMPANION TO ClimatePros Stream 1 Scope Document v2.0 (July 21, 2026)



Document Control

Item	Detail
Document	Stream 1 — Unified Architecture
Version	1.0 — Draft for Client Review
Issued	July 28, 2026
Prepared by	techjays
Classification	Confidential — ClimatePros / techjays
Companion to	Stream 1 Scope Document v2.0 (July 21, 2026) — that document defines <i>what</i> is in scope; this one defines <i>how</i> it is built.
Source	Generated from the stream1-unified-architecture design set (sections 1–12)

How to Read This Document

- Sections are numbered 1–12 and match the live architecture document one-to-one. Cross-references appear as §N — for example §6 Core Logic.
- **Cloud placement rule, applied throughout:** anything AI-specific runs on **Azure** — the FastAPI AI server, its MongoDB, and the vector store. Everything else runs on **AWS**: the .NET APIs on EC2, SQL Server, the ElastiCache for Redis dispatch queue, the ECS Fargate workers, and Amazon SNS push. Models stay **on-prem** on ClimatePros hardware. The single cross-cloud hop is the AI request itself (AWS → Azure).
- **Use Case A involves no LLM.** It is a deterministic weighted-sum ranking and must stay explainable. Use Cases B and C are AI-assisted, and in both the technician can always override the output.
- Diagrams appear as **Figure N** placeholders for screenshots from the live HTML document; each carries a caption describing what the figure shows, so the text stands alone without it.
- Formulas, schemas, scoring constants, API parameters and cost line items are reproduced at full fidelity from the source design, not summarised.
- Terminology corrected during conversion is listed in **Appendix A**; open items needing ClimatePros confirmation are listed in **Appendix B**.

Approval

Approved for ClimatePros: _____ Date: _____

Approved for techjays: _____ Date: _____

Contents

01 Stream 1 Overview	4
Auto-Dispatch — finding the right technician	4
Job Summarisation Copilot — changing the job status	4
Technician Parts Finder — finding the part that isn't on the truck	5
02 High-Level Architecture	6
2a · Unified system diagram	6
2b · Detailed architecture per use case	9
03 Technology Stack	12
Dispatch Worker Stack A	14
Central AI Service Stack B C	14
04 Infrastructure	16
FieldJetX Platform	16
Auto-Dispatch tier + central AI server	16
Environments	17
Compute sizing	17
Network topology	18
05 Operational Flows	20
Auto-Dispatch — Assignment Loop	20
Job Summarisation — Technician Flow	23
Parts Finder — Find & Allocate	28
06 Core Logic (per use case)	31
Auto-Dispatch — Ranking Algorithm	31
Job Summarisation — Summarise Pipeline & Retrieval	32
Parts Finder — Matching & Sourcing Logic	34
07 Data & Schema	36
Auto-Dispatch — Amazon ElastiCache for Redis Data Structures	36
Job Summarisation — the AI MongoDB Collections	36
Parts Finder — Data Model	37
08 API Reference	39
Auto-Dispatch — API Reference	39
Job Summarisation — API Reference	40
Technician Parts Finder — API Reference	41
09 Observability & Evals	46
Unified Observability — OpenTelemetry Tracing + CloudWatch / Azure Monitor Logs	46
Evals — the Feedback Loop	47
10 Cost Projection	48
Compute Sizing	48
Cost Formulas	49

Unified Monthly Cost by Scenario	49
Database & Cross-Cloud Egress	50
11 Onboarding & Fault Tolerance	51
How every rollout runs	51
Auto-Dispatch	51
Job Summarisation	53
Parts Finder	54
Environments	17
12 Non-goals & PII	56
Non-goals	56
PII inventory & data handling	57
Appendix A — Corrections applied during conversion	59
Appendix B — Open items	62

01 Stream 1 Overview

ClimatePros runs commercial refrigeration & HVAC field service: ~900–1,000 technicians, 46 branches, 10,000+ sites. Work orders arrive automatically. Everything after that still runs on a phone call and a technician's memory. Three use cases, three bottlenecks, one job.

Where AI sits: Use Case A is a deterministic algorithm — no LLM. It replaces a decision a dispatcher already makes by hand, so it must be explainable and repeatable. Use Cases B and C use AI, and in both the technician can always override it.

~900–1,000

TECHNICIANS

46

BRANCHES

10,000+

SITES

A ALGORITHM • NO LLM

Auto-Dispatch — finding the right technician

WEB PORTAL • DISPATCHER

A dispatcher has a job ready and ~40 candidate techs in the region. The four things that decide the right one — how far, how familiar with the site, how soon free, how urgent — are each buried behind separate clicks or held in the dispatcher's head. Confirming one means dialling candidates one at a time, ~2–3 min each; no answer → call the next.

Why it hurts: two dispatchers per branch are the throughput ceiling — more job volume means more headcount, not faster dispatch.

We solve: rank on those same four factors; push an accept/decline offer that cascades automatically. No LLM — every assignment explainable.

B AI • TECH OVERRIDES

Job Summarisation Copilot — changing the job status

MOBILE • TECHNICIAN AT END OF JOB

Every job ends with a mandatory status change, and behind it sits a wall of forms plus two free-text notes — one for the customer, one for the office. Done properly it takes 10–15 min, often in a machine room with no signal where voice dictation is only ~30% accurate.

Why it hurts: so the tech skimps — ~40% of 16.3M notes are ≤40 chars, the most common note is literally "checked history". Downstream the office can't bill or route ("Replaced motor" — but no motor in the parts list).

We solve: restructure rough words into a clean customer/office note and suggest form values from similar jobs; flag what's missing. Never fabricates, never blocks Save. Target: 10–15 min → ~15 s.

C AI • TECH OVERRIDES

Technician Parts Finder — finding the part that isn't on the truck

MOBILE • TECHNICIAN ON-SITE

On site the tech needs a part. Three sources the app already answers — truck, nearby techs, warehouse. The fourth, a third-party supplier, is a phone call: Google search, 1–3 calls, hope they answer.

Why it hurts: the same physical part carries a different number at every vendor, and the supplier API matches exact part number only — no fuzzy logic. A human bridges that gap by phone under emergency pressure; miss it and it's a return dispatch — a second truck roll, job can't finish today.

We solve: AI suggests the likely part; one search shows where it is across all four sources with location and distance. No price ranking, no ordering.

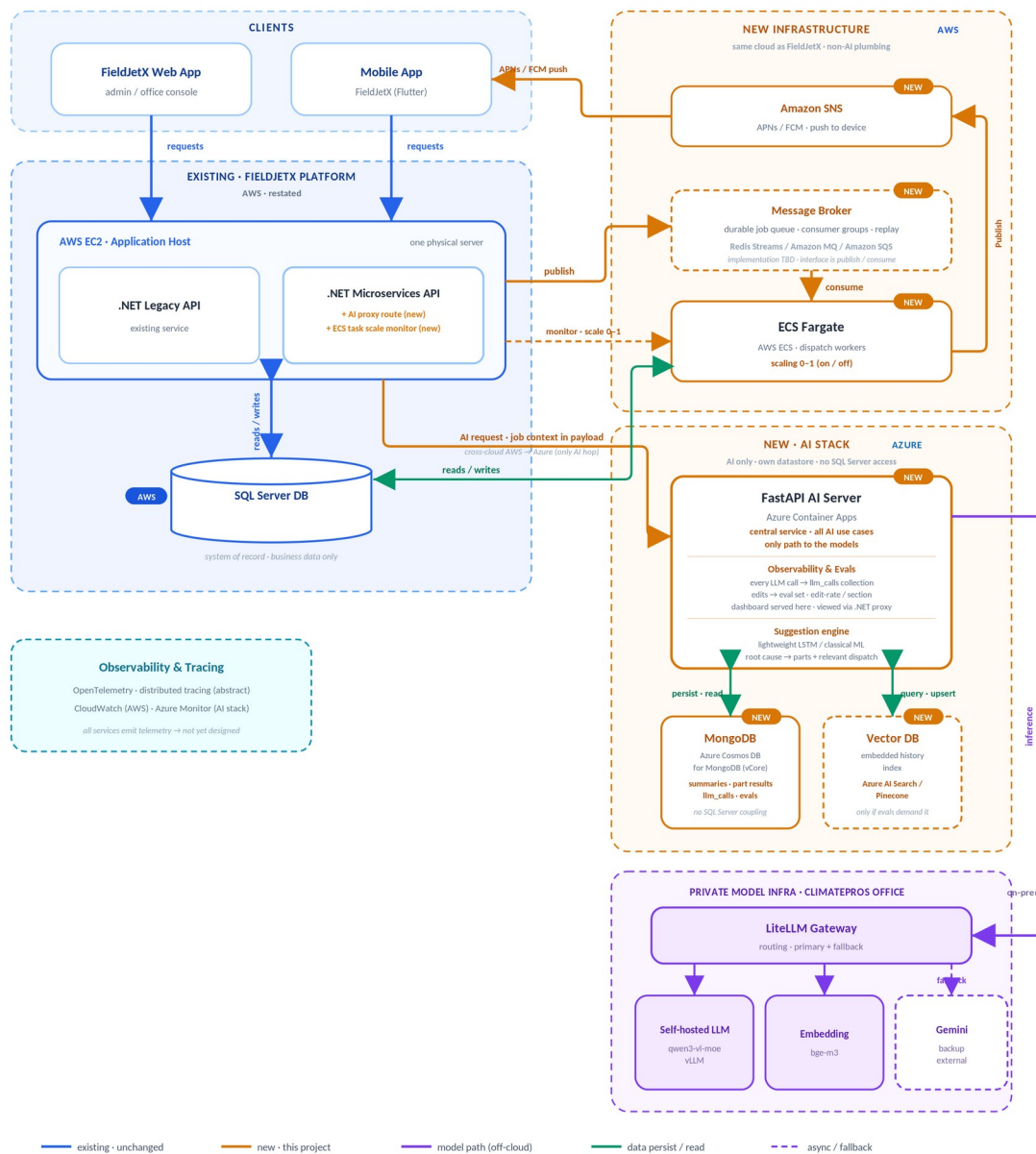
Work order in → (A) the right tech is offered the job on the web portal → tech arrives on site → (C) the missing part is found on mobile without a phone call → (B) the closeout note writes itself. Full step-by-step diagrams for all three: see §5 Operational Flows.

02 High-Level Architecture

2a · Unified system diagram

FIGURE 1

Unified system diagram · ClimatePros Stream 1



Six zones: clients; the existing FieldJetX platform on AWS; new AWS infrastructure (queue, workers, push); the new Azure AI stack (FastAPI AI server, MongoDB, vector store); the private on-prem model infra; and the cross-cutting observability layer. Live version: stream1-unified-architecture.html.

Everything runs on AWS alongside the existing FieldJetX platform — except the AI stack (FastAPI AI server, its own MongoDB, vector DB), which runs on Azure. The AI server is the single cross-cloud hop, and it holds no SQL Server connection: it owns its own datastore.

The diagram groups the system into six zones — clients, the existing FieldJetX platform (AWS), the new AWS infrastructure (queue, workers, push), the new Azure AI stack (AI server + its own datastore + vector DB), the private on-prem models, and a cross-cutting observability layer. What follows is the overall flow: which entity talks to which, and why. Per-use-case wiring (queues, locks, schemas, ranking, scoring) is detailed in §2b — this section stays deliberately high-level.

Entry · single front door

Web App + Mobile App → AWS EC2 (.NET APIs)

Both clients speak only to the existing .NET APIs on the EC2 application host over HTTPS. Neither the new infrastructure nor the models are ever reached directly — the legacy platform stays the single entry point and gains just two new additions: an AI proxy route and an ECS task-scale controller (spins workers up/down).

Data · two stores, cleanly split

EC2 ↔ SQL Server (AWS) ↔ ECS workers | FastAPI ↔ MongoDB (Azure)

SQL Server stays the system of record for business data — the EC2 .NET APIs and the new ECS dispatch workers read/write it directly, all inside AWS. The FastAPI AI server never connects to SQL Server: it owns a separate MongoDB (Azure Cosmos DB for MongoDB) for summaries, part-search results, llm_calls traces and eval data. Any business context it needs arrives in the request payload from the .NET AI proxy.

Dispatch · Use Case A · no LLM

.NET → ElastiCache → ECS Fargate → SNS → Mobile

When auto-dispatch is on, the .NET API publishes the job to ElastiCache for Redis and the scale monitor spins ECS Fargate workers 0→1. Workers consume the stream, rank technicians, and push accept/decline offers out through Amazon SNS (APNs/FCM) to the technician's phone. Entirely on AWS — no cross-cloud hop. The reply path returns via the .NET API. Deterministic — no model involved.

AI · Use Cases B & C

.NET (AWS) → FastAPI AI Server (Azure) → Vector DB / MongoDB

For summarisation (B) and parts finding (C), the .NET AI proxy forwards the request — with the job context in the payload — across to the FastAPI AI Server on Azure, the single central AI service and the only path to the models. It queries/upserts the Vector DB for historical retrieval and persists its outputs to its own MongoDB. This AWS→Azure call is the sole cross-cloud dependency; the AI stack is self-contained behind it.

Models · private, on-prem

FastAPI → LiteLLM Gateway → self-hosted LLM / embedding / Gemini

All inference goes through the on-prem LiteLLM Gateway, which routes to the self-hosted qwen3-vl-moe (vLLM) and bge-m3 embedding, with Gemini as external fallback. No service calls a model directly — LiteLLM is the sole gate, keeping data inside ClimatePros infra.

Observability · cross-cutting

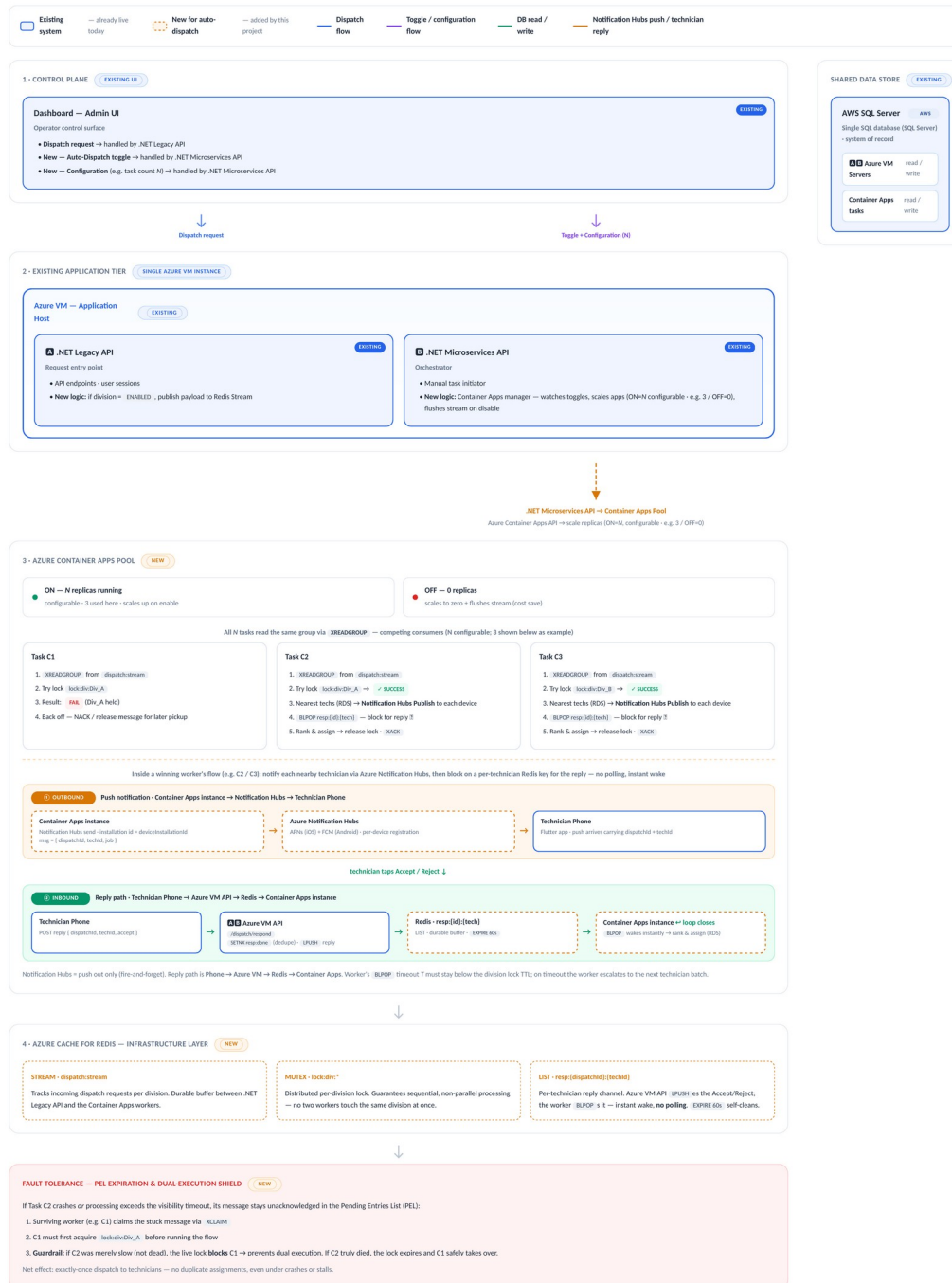
all services → OpenTelemetry / CloudWatch + Azure Monitor

Every service emits traces and logs to a shared observability layer (OpenTelemetry) — CloudWatch on the AWS side, Azure Monitor for the AI stack.

2b • Detailed architecture per use case

FIGURE 2

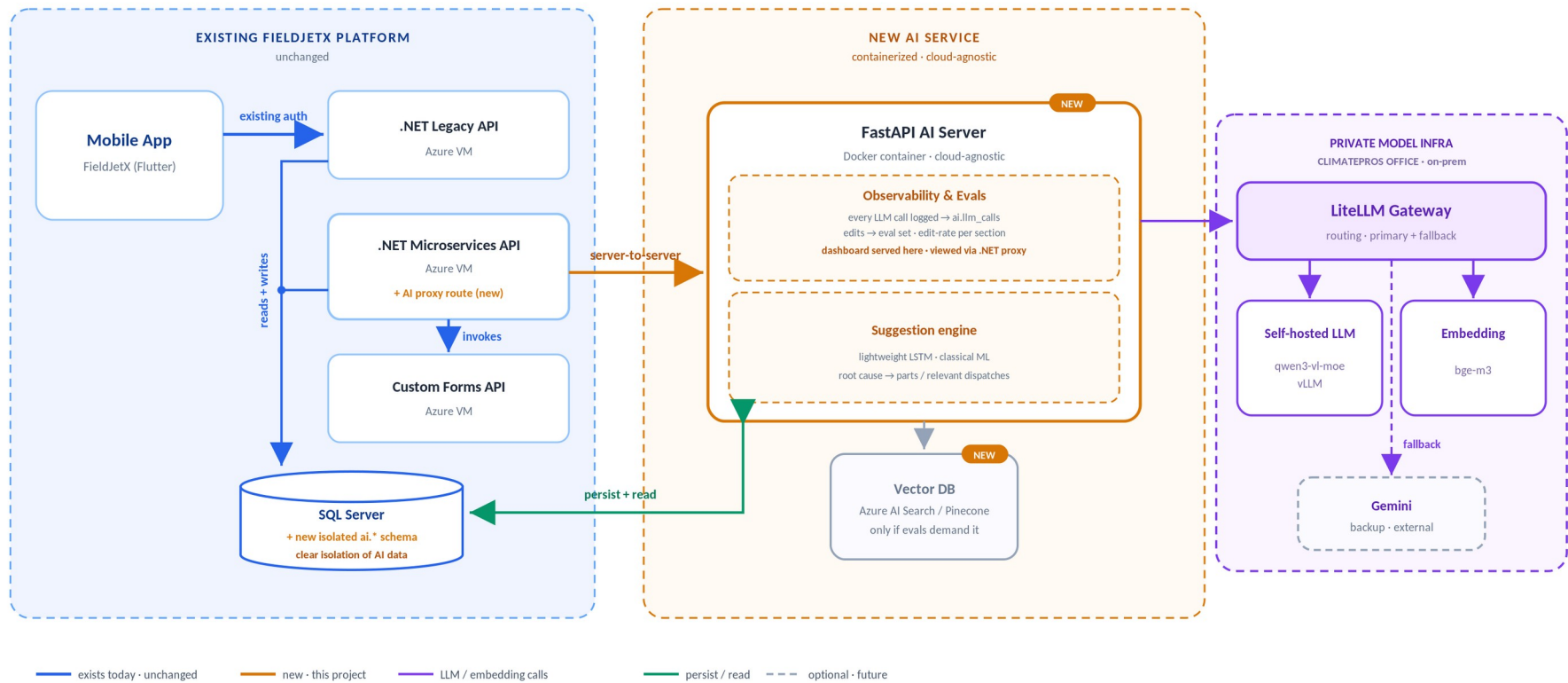
Use Case A — detailed architecture (Auto-Dispatch)



Blue = existing · unchanged; amber dashed = new · this project; purple = toggle/config flow; green = DB read/write. Entirely within AWS. Live version: stream1-unified-architecture.html.

FIGURE 3

Use Cases B & C — detailed architecture (shared AI service)



Blue = existing · unchanged; amber dashed = new · this project; purple = LLM/embedding calls (client-hosted, off-cloud); green = persist/read. Grey dashed = optional vector index. Live version: stream1-unified-architecture.html.

A A adds: durable dispatch queue + ECS Fargate worker + push-offer cascade.

Use Case A architecture: blue = existing · unchanged, amber dashed = new · this project, purple = toggle/config flow, green = DB read/write. All of Use Case A runs on AWS: the .NET Legacy and Microservices APIs on AWS EC2, the dispatch workers on AWS ECS Fargate, the durable queue on Amazon ElastiCache for Redis, push through Amazon SNS, and AWS SQL Server as the system of record. No part of A touches Azure — there is no LLM in this use case.

B C B & C add: central FastAPI AI server + vector history index. B runs the summarise pipeline; C the four-source part search + fuzzy matching — same wiring, different pipeline.

One diagram serves both B and C — they share the same detailed architecture (central FastAPI AI server + vector history index, same Azure/on-prem model path). Only the pipeline differs, detailed in §6. Blue = existing · unchanged, amber dashed = new · this project, purple = LLM/embedding calls (client-hosted, off-cloud), green = persist/read. The vector history index is an optional future add-on (grey dashed), added only if evals demand it. The AI service is a portable Docker container; the existing platform is restated on AWS EC2. The FastAPI AI server and its datastores are the only Azure components in this diagram.

AI data is self-contained. The FastAPI AI server owns its own MongoDB (Azure Cosmos DB for MongoDB, vCore) and holds no SQL Server connection at all — it persists its own outputs (summaries, part-search results, LLM-call traces and eval data) in its own collections. Any business context it needs arrives in the request payload from the .NET AI proxy, so the AI datastore is never coupled to the legacy relational schema.

03 Technology Stack

Ten layers, three use cases. The existing platform pieces — client apps, the .NET API host, SQL Server, observability — are shared: reused as-is, not rebuilt. A's new tier — durable dispatch queue, worker pool, push — stays on AWS alongside the existing platform; B and C's new tier — a central AI service and vector store — is the one part of Stream 1 that runs on Azure, reached by a single cross-cloud hop from the AWS side.

Layer	A Auto-Dispatch	B Job Summarisation	C Parts Finder
Client	FieldJetX Web App (Angular) existing + Mobile App (Flutter) existing Dispatcher console (web) + technician accept/decline offers (mobile)	FieldJetX Mobile App (Flutter) existing Technician close-out wizard	FieldJetX Mobile App (Flutter) existing Technician on-site search
App / API host	.NET Legacy API + .NET Microservices API on AWS EC2 existing One shared application host across all three use cases. The Microservices API gains a new AI-proxy route (B, C) and a new ECS Fargate scale-monitor route (A).		
Central AI service	N/A deterministic — no LLM	FastAPI on Azure Container Apps new Single entry point for both B and C; stateless, horizontally scalable containers; the only path to the models.	
Workers	AWS ECS Fargate new Async worker pool · scales 0–1 on toggle · 20 concurrent dispatches per task	N/A	N/A
Message Broker	Amazon ElastiCache for Redis new Redis Streams (XREADGROUP / XACK / XCLAIM) · division locks · per-technician reply BLPOP · decided implementation for the durable dispatch queue	N/A	N/A
Vector store	N/A	Azure-hosted vector store experimental Cross-site similar-incident retrieval. Built only if the dataset proves it beats the deterministic baseline — decided from the data.	Azure-hosted vector store experimental Fuzzy part-number matching via embedded dispatch-parts history + cosine similarity. Built only if the dataset proves it beats exact/keyword lookup — decided from the data.

Layer	A Auto-Dispatch	B Job Summarisation	C Parts Finder
Push	Amazon SNS new APNs (iOS) + FCM (Android) · one Publish call per device endpoint	N/A	N/A
Database	AWS SQL Server AWS System of record. A reads/writes dispatch & assignment tables directly from AWS EC2 and AWS ECS Fargate — no cross-cloud hop for this data. B and C add no tables to SQL Server at all: they persist their AI output to the AI server's own MongoDB on Azure, and read business context only via the payload the .NET AI proxy sends. Stream 1's one cross-cloud hop is the AI request itself (AWS → Azure), not this database (see §2 cloud placement).		
Models	N/A no LLM in this use case	LiteLLM gateway (routing · primary + fallback) · self-hosted qwen3-vl-moe (vLLM) generation · bge-m3 embedding · Gemini fallback on-prem · unchanged Client's private hardware; dispatch/job text and images never leave client infrastructure. Any model or provider change must pass the eval set first.	
Observability	OpenTelemetry + CloudWatch (AWS) + Azure Monitor (AI stack) new Every service — AWS EC2 host, ElastiCache-fed workers, ECS Fargate, the Azure AI server, vector store calls — emits traces and logs via OpenTelemetry, viewed through CloudWatch on the AWS side and Azure Monitor for the AI stack.		

New non-AI infrastructure — the message broker, the worker pool, and push — stays on AWS, matching the existing FieldJetX platform. Only the AI stack (the central AI service, its MongoDB, and the vector store) runs on Azure; that AI-request hop is Stream 1's sole cross-cloud dependency. AWS SQL Server is unaffected either way — it was always AWS and is read only by AWS-side compute.

Dispatch Worker Stack A

Library-level detail for the Auto-Dispatch async worker (the Workers / Message Broker / Push rows above). Fully async top-to-bottom — no thread pools, no blocking calls in the event loop.

1 Core Runtime

- Python 3.11+
- asyncio event loop drives all worker tasks
- asyncio.Semaphore for concurrency control

2 Push — Amazon SNS

- aioboto3 async client against the SNS Publish API
- IAM role credentials — no manual token handling
- One Publish call per platform endpoint ARN (APNs / FCM)

3 Messaging & Sync — Amazon ElastiCache for Redis

- redis.asyncio client against the ElastiCache endpoint
- Input: consumes job tasks via Redis Streams (XREADGROUP)
- Wait-state: unblocked via Pub/Sub or a secondary stream when the mobile app's HTTP reply reaches the API host

4 External Requests

- httpx with built-in async connection pooling for 3rd-party API calls

Central AI Service Stack B C

Library-level detail for the shared FastAPI AI service (the Central AI service row above) — the single entry point for both B and C, and the only path to the models.

1 Core Runtime

- Python 3.11+
- FastAPI async framework · Uvicorn ASGI server
- pydantic request/response schemas

2 Model Access

- httpx async client to the LiteLLM gateway
- Generation via qwen3-vl-moe (vLLM) · embeddings via bge-m3
- Gateway handles primary + fallback routing (Gemini fallback)

3 Retrieval experimental

- Async client to the Azure-hosted vector store
- Cosine-similarity search over bge-m3 embeddings
- Built only if it beats the deterministic baseline — decided from the data

04 Infrastructure

What Stream 1 already has vs. what it introduces — the same §3 layers, regrouped by existing-platform vs. new-build. The existing platform and all new non-AI plumbing stay on AWS, aligned with FieldJetX; only the AI stack (FastAPI AI server + its own datastores) runs on Azure, with the models on-prem. See the callout below.

EXISTING • REUSED AS-IS

FieldJetX Platform

MOBILE APP — FIELDJETX (FLUTTER) • TECHNICIAN-FACING (B, C)

- FieldJetX Web App — admin / dispatcher console (A)
- AWS EC2 — one host running both .NET services AWS
- .NET Legacy API — existing service on the EC2 host AWS EC2
- .NET Microservices API — existing service on the EC2 host; gains new routes for A/B/C AWS EC2

AWS SQL SERVER — SYSTEM OF RECORD • BUSINESS DATA ONLY AWS

NEW • INTRODUCED BY THIS PROJECT

Auto-Dispatch tier + central AI server

- Message Broker — Amazon ElastiCache for Redis, using Redis Streams (consumer groups + replay) as the durable dispatch queue for Auto-Dispatch (A) AWS
- ECS Fargate — Auto-Dispatch workers — scale 0-1 on toggle; consume queued dispatches, read/write SQL Server AWS ECS

Amazon SNS — outbound push to technician phones (APNs / FCM); worker → SNS → Mobile App AWS

- Azure Container Apps — AI Server (FastAPI) — central AI service for B & C; the only component that reaches the models. Two subsystems ride inside it: Observability & Evals — every LLM call → llm_calls • technician edits → eval set • dashboard served here, viewed via the .NET proxy Suggestion engine — lightweight LSTM / classical ML • root cause → parts + relevant dispatch Azure
- MongoDB — AI-owned datastore: summaries • part results • llm_calls • evals. No SQL Server coupling. Azure Cosmos DB (vCore)
- Vector store — embedded dispatch-parts history index (C) • optional future similar-incident index (B) • only if evals demand it Azure AI Search / Pinecone
- .NET additions — AI-proxy route (B, C) + ECS task scale-monitor route (A) on the existing Microservices API AWS EC2

- LiteLLM Gateway + self-hosted LLM (qwen3-vl-moe / vLLM) + embedding (bge-m3) — private model infra, client hardware on-prem
- Google Gemini — managed fallback only, not primary external

Observability & Tracing — OpenTelemetry traces + logs → CloudWatch (AWS) + Azure Monitor (AI stack) abstract

2	NEW COMPUTE TIERS	ECS Fargate workers (0-1) + Container Apps AI server
1	NEW MESSAGE BROKER	Amazon ElastiCache for Redis (Redis Streams)
2	NEW DATA STORES	MongoDB + vector store (conditional)
3	OFF-CLOUD MODEL ENTITIES	LiteLLM + self-hosted LLM + embedding
1	EXTERNAL BACKUP	Google Gemini (fallback only)

Environments

ClimatePros already runs Dev / Staging / Prod. Stream 1 mirrors all three. New AWS tiers (message broker, dispatch workers, push) and new Azure tiers (the AI server, its MongoDB, and the vector store) — plus the on-prem model link — are stood up per environment; the AWS SQL Server system of record is shared — lower environments point at a snapshot or a restricted replica, never live prod data (see §12 PII).

Environment	Purpose	What differs from Prod
Dev	Integration & feature work provisioned alongside Staging + Prod	Lowest replica caps; Gemini-only models (no on-prem link); points at a SQL snapshot
Staging	Pre-prod validation & evals mirrors Prod topology	Smaller replica ranges; restricted SQL replica; full model stack for eval fidelity
Prod	Live — 900-1,000 technicians, 46 branches full scale + HA	—

Compute sizing

New compute tiers — AWS ECS Fargate for dispatch workers, Azure Container Apps for the AI server — run small and scale horizontally, not vertically. Baseline instance size is 0.5 vCPU / 1 GB RAM across both, which handles 60+ concurrent dispatch operations per instance comfortably; load is met by adding replicas/tasks, not bigger boxes.

Component	Instance size	Scaling	Notes
Dispatch workers (ECS Fargate) AWS	0.5 vCPU / 1 GB	Scale 0–1 on Auto-Dispatch toggle	60+ concurrent dispatches per instance; one instance covers current volume
AI server (FastAPI) Azure	0.5 vCPU / 1 GB	Scale 1–N on request concurrency	Central B & C service; min 1 replica avoids cold-start on the model path
Amazon ElastiCache for Redis AWS	Managed tier	Fixed; sized to queue depth	Dispatch queue must not evict — persistence on, no-eviction policy
Vector store Azure	Managed / index-sized	Grows with parts-history corpus	Embedded dispatch-parts index (C); optional similar-incident index (B)

Network topology

The existing platform and all new non-AI infrastructure sit inside a single AWS VPC, matching FieldJetX: EC2 (the .NET hosts) and ECS Fargate (dispatch workers) in an apps subnet, and ElastiCache for Redis and SQL Server reached over private subnets — neither exposes a public IP, so all worker ↔ broker and worker ↔ database traffic stays inside the VPC. The only public entry point is the .NET API, fronted by an AWS load balancer / API gateway where WAF filtering and TLS termination happen; the mobile app (B, C) and web console (A) reach the API there.

A separate Azure VNet holds only the AI stack: Container Apps (the FastAPI AI server) in an apps subnet, with MongoDB and the vector store reached over private endpoints in a data subnet. The one cross-cloud path in Stream 1 is the .NET AI-proxy route on AWS calling the FastAPI AI server on Azure — technician phones never reach either side directly, and the AI server never reaches AWS SQL Server. From there, the AI server has two private egress paths of its own: the on-prem model stack (LiteLLM + self-hosted models) over a VPN / private link to the ClimatePros datacenter, and a fallback to Google Gemini over outbound HTTPS. That gives three trust boundaries to enforce: Internet→AWS (load balancer / WAF), AWS→Azure (the single AI-request hop — VPN/peering target state, public endpoint + IP allow-list as fallback), and Azure→on-prem (VPN / private link).

Cross-cloud dependency:

THE SEAM ISN'T THE DATABASE — • AWS SQL SERVER

is read and written only by AWS-side compute (EC2 and ECS Fargate), so Auto-Dispatch (A) never leaves AWS. The one cross-cloud hop is the AI request itself: when a technician triggers a summarisation (B) or parts search (C), the .NET AI-proxy route on AWS EC2 calls the FastAPI AI Server on Azure Container Apps, carrying the job context in its payload, and gets a response back.

- Network path: target state is a site-to-site VPN or VPC-to-VNet private peering between the AWS account and the Azure subscription; until that's provisioned, the fallback is a public HTTPS endpoint on the AI server, locked down by IP allow-listing / mTLS.

- Added latency: only the B/C request path pays inter-cloud round-trip time — once per summarisation or parts search, not per SQL call — and this has to be budgeted into the P95 targets in §9 Observability & Evals.
- Cost: AWS bills egress on the .NET proxy's outbound call to Azure; this is one request/response pair per B/C call, not a per-row data-transfer cost, and belongs in §10 Cost Projection.
- Single seam: this AI-request hop is the only place Stream 1 crosses clouds — SQL Server, the message broker, the dispatch workers, and push all stay inside AWS. If the link goes down, B and C fall back to their existing manual path (forms without AI assist, no parts search); Auto-Dispatch (A) is entirely unaffected since it never touches Azure.

05 Operational Flows

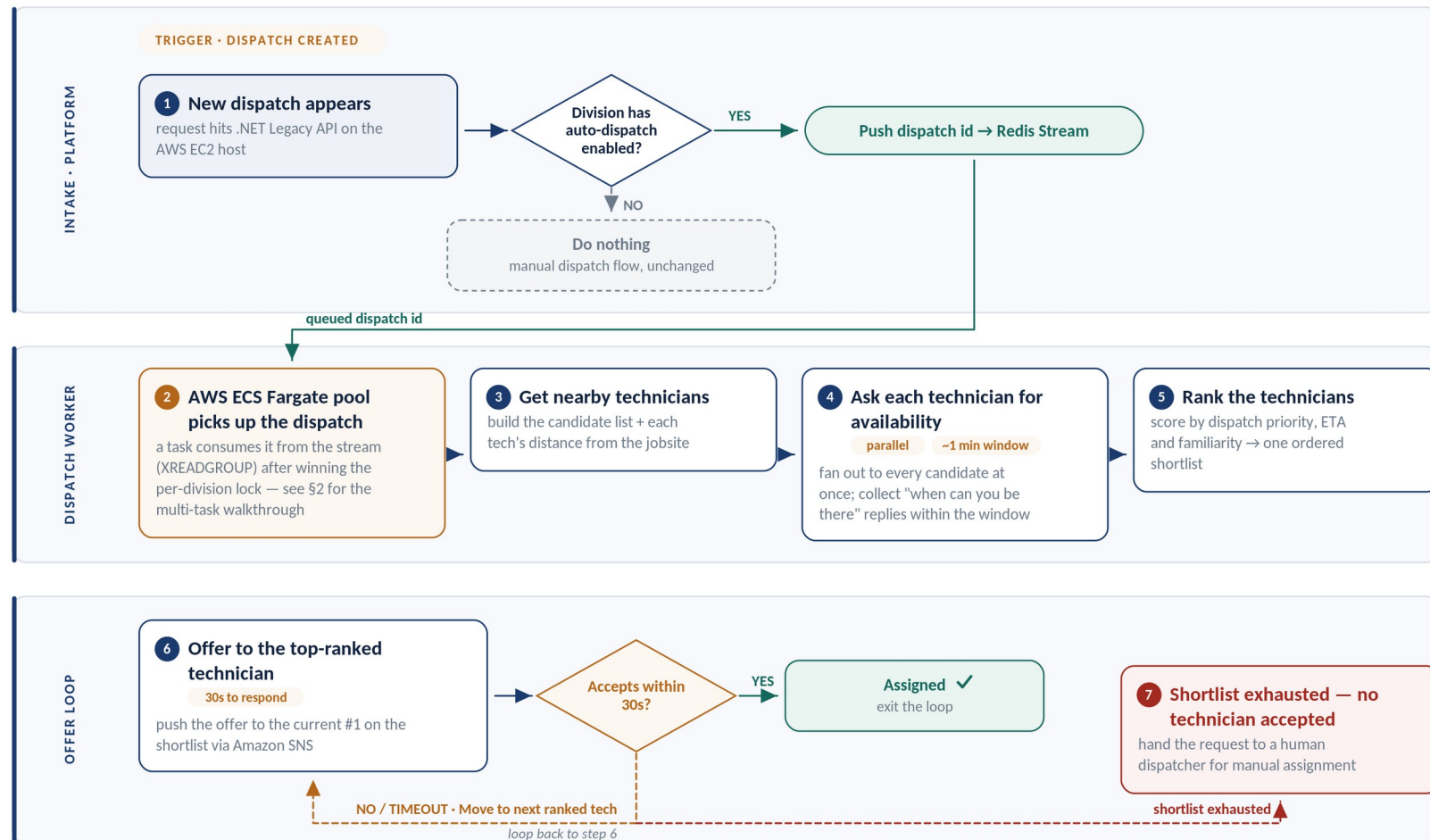
The request/response mechanics for each use case, trigger to outcome. A is a closed-loop dispatch cascade with a hard timeout and human escalation; B is a technician-facing UX flow, shown both before and after the copilot, plus the background flywheel that feeds each visit's data into the next; C is the ordered, multi-source parts search that backs the mobile app's "source & allocate" step. Companion to §2 Architecture and §3 Technology Stack — component-level detail (Redis Streams, division locks, ECS Fargate task walkthroughs) lives there; this section is the process view.

A Auto-Dispatch — Assignment Loop

One dispatch, start to finish: a new job only enters the queue for divisions with auto-dispatch turned on, then works down a ranked shortlist until a technician accepts or the shortlist runs out. The admin Enable/Disable toggle is covered by the scale ON/OFF walkthrough in §2.

FIGURE 4

Use Case A — assignment loop (trigger to outcome)



The ranked cascade with a 30-second offer window per technician and escalation to a human dispatcher when the shortlist is exhausted. Steps are written out below. Live version: stream1-unified-architecture.html.

ASSIGNMENT SEQUENCE • TRIGGER TO OUTCOME

#	Step	What happens
1	New dispatch appears	request hits .NET Legacy API on the AWS EC2 host
◇	Division has auto-dispatch enabled?	NO → Do nothing (manual dispatch flow, unchanged) · YES → Push dispatch id → Redis Stream
2	AWS ECS Fargate pool picks up the dispatch	a task consumes it from the stream (XREADGROUP) after winning the per-division lock — see §2 for the multi-task walkthrough
3	Get nearby technicians	build the candidate list + each tech's distance from the jobsite
4	Ask each technician for availability parallel ~1 min window	fan out to every candidate at once; collect "when can you be there" replies within the window
5	Rank the technicians	score by dispatch priority, ETA and familiarity → one ordered shortlist
6	Offer to the top-ranked technician 30s to respond	push the offer to the current #1 on the shortlist via Amazon SNS
◇	Accepts within 30s?	NO / TIMEOUT → Move to next ranked tech (∪ loop back to step 6) · YES → Assigned ✓ (exit the loop)
7	Shortlist exhausted — no technician accepted	hand the request to a human dispatcher for manual assignment

Timeout rule: the worker's reply-wait timeout has to stay below the division lock's TTL. On timeout the worker escalates to the next technician on the shortlist; the division lock is never lost mid-flight.

Amazon SNS round-trip

The mechanics behind step 6's push — fire-and-forget out, a different path back in. Component-level diagram: §2 · Use Case A · step 3.

#	Step	What happens
1	Worker publishes the offer	The winning ECS Fargate worker calls the Amazon SNS Publish API with the technician's platform endpoint ARN and the offer payload {dispatchId, techId, job}.
2	Push is delivered	Amazon SNS fans the message out to the platform application endpoint through APNs (iOS) or FCM (Android) straight to the technician's phone.
3	Technician responds	Tapping Accept/Reject in the FieldJetX app POSTs the reply to the AWS EC2 API, which de-dupes with SETNX resp:done and LPUSH es it onto a per-technician Redis reply list.
4	Worker wakes instantly	The same worker that sent the offer is already blocked on BLPOP for that key — no polling, no delay.
5	Rank & assign	On Accept the worker writes the assignment to SQL Server, releases the division lock, and XACK s the stream message — closing the loop.

B Job Summarisation — Technician Flow

The same close-out visit, shown twice: the manual process technicians use today, and the same steps with the copilot layered on top — plus the background flywheel that turns every clean ticket into better history for the next technician.

Today · without AI

How a service visit works in the mobile app today. It works end to end, but leaves value on the table at three points: technicians arrive without the site's history, every record is typed by hand, and quality issues surface only after the technician has left. Red marks where those costs arise.

FIGURE 5

Use Case B — the close-out visit today, without AI



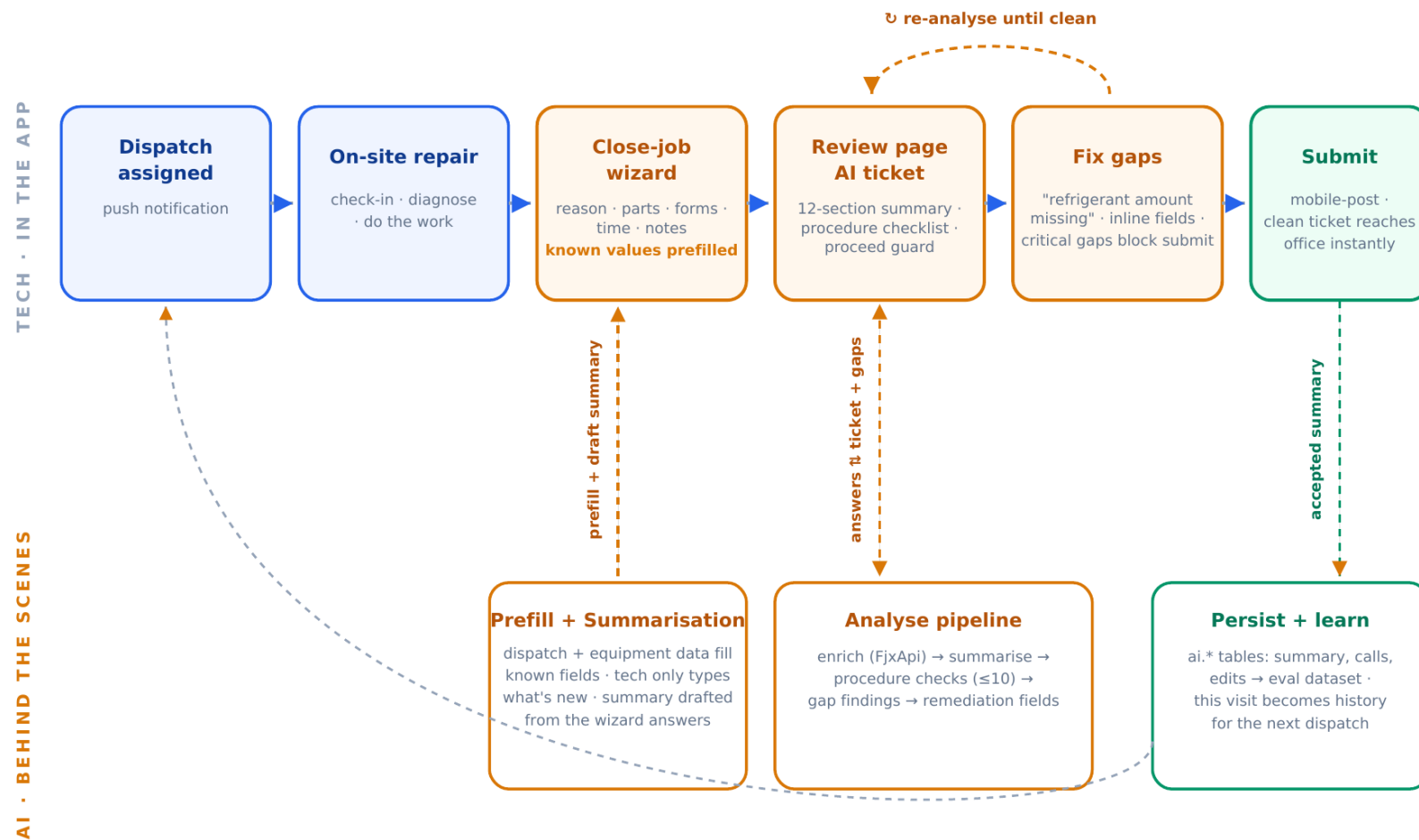
Left column = technician steps in the app; right column = where cost arises today (goes in cold, quality varies by typist, no validation before submit, problems found when expensive). Live version: stream1-unified-architecture.html.

With the AI Copilot

The same visit with the copilot in place. The technician's journey doesn't change; the AI works alongside it — preparing a briefing before arrival, drafting the paperwork, and checking the ticket before submit. Blue = technician action • amber = AI-assisted step.

FIGURE 6

Use Case B — the same visit with the AI Copilot



the flywheel: every clean ticket becomes better history for the next dispatch

Left column = technician actions (unchanged journey); right column = AI work behind the scenes (briefing builder, prefill + summarisation, analyse pipeline, persist + learn). Blue = technician action; amber = AI-assisted step. Live version: stream1-unified-architecture.html.

Operational flow — the flywheel

Three flows drive the copilot day to day: building the site briefing at assignment time, the summarise-and-review loop when the technician closes the job, and the persist-and-learn path after submit.

Site Briefing

TRIGGER • DISPATCH ASSIGNED

- 1 Assignment event received Use Case A (or manual dispatch) assigns the job; the briefing builder starts in the background
- 2 Gather this site's history keyed SQL query over past visit digests: same customer, same site
- 3 Find 2–3 similar past issues deterministic SQL retrieval (same equipment type + problem code, most recent first); customer procedures fetched live, never cached
- 4 LLM writes the briefing card “past techs found X • issue recurred • try Y first” • cached in MongoDB

Opens instantly when the tech taps the dispatch

Summarise & Review

TRIGGER • TECH REACHES REVIEW STEP

- 1 App compiles the wizard payload zero extra API calls • POST /api/v1/summarise through the .NET proxy
- 2 Enrich, prompt, generate parallel fail-soft reads → versioned ticket standard → the LLM drafts the 12-section ticket + procedure checks

CRITICAL GAPS • FOUND? • YES • NO • FIX INLINE

🔄 re-analyse until clean

TICKET CLEAN • SUBMIT ENABLED • PERSIST & LEARN

TRIGGER • TECH TAPS SUBMIT

- 1 Job write-back, unchanged dispatch • parts • time • notes flow through the existing mobile-post path
- 2 AI output persisted accepted summary → the summaries collection; every call already logged in the llm_calls collection
- 3 History refreshed the closed dispatch immediately extends the SQL history and co-occurrence counts the next briefing and suggestions read; no scheduler, no pipeline
- 4 Corrections become ground truth any edits land in the summary_edits collection and feed the eval set

This visit becomes the next tech's briefing

The flywheel: the three flows chain into a loop — what Persist & Learn stores is exactly what Site Briefing retrieves for the next assignment, so every clean ticket makes the next technician's briefing better.

C Parts Finder — Find & Allocate

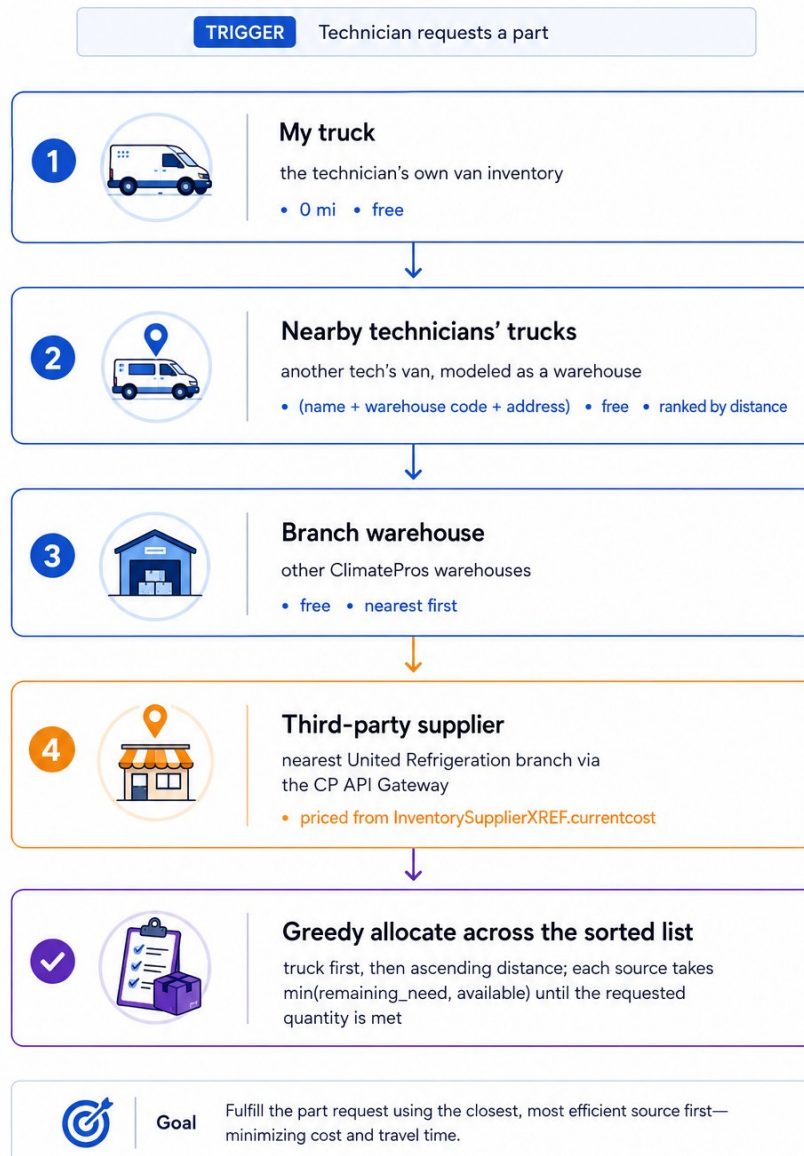
For every part the technician needs, the app gathers every place it can be collected and orders them so the closest free stock comes first — the technician's own truck (on hand, zero distance), then other technicians' vans and branch warehouses (own stock, nearest first), and finally the United Refrigeration vendor. The needed quantity is then split greedily down that list: each source contributes what it has until the quantity is met.

FIGURE 7

Use Case C — four-source find and greedy allocate

Four-Source Find → Allocate

Determine and allocate the nearest and most efficient source to fulfill a technician's parts request.



Truck → nearby technicians' trucks → branch warehouse → third-party supplier, then greedy allocation down the sorted list. Steps are written out above.

A van is a warehouse. Per operations, there is no separate "nearby technician" tier — another tech's van is modelled as a warehouse, which keeps the sourcing model to three underlying data tiers (truck / warehouse / vendor) while the technician-facing flow above still reads as four distinct sources.

Vendor lookup fails → drop the tier, not the request. If the United Refrigeration lookup fails, the vendor tier is dropped rather than failing the whole request — the technician still sees every own-stock option (truck + warehouse). See §11 Onboarding & Fault Tolerance.

06 Core Logic (per use case)

The algorithms and pipelines behind the flows in §5, lifted at full fidelity — every formula, weight, table and worked example intact, not condensed: how A ranks a shortlist of technicians, how B turns wizard answers into a 12-section ticket (plus the deterministic retrieval and measured baseline it stands on), and how C predicts parts by retrieval and sources them across three tiers.

A Auto-Dispatch — Ranking Algorithm

This is the detail behind step 3 (“Rank the technicians”) of the assignment loop in §5. It turns each candidate's availability, distance, and visit history into a single ordered shortlist. One formula covers every case — its behavior is tuned by a single weight w chosen from the job's dispatch type.

Inputs

- free_i — minutes until technician i is available (time left on current job)
- dist_i — miles from technician i to the jobsite
- visits_i — prior visits by technician i to this customer (familiarity)
- AVG_SPEED — average truck speed in mph, a configurable constant

DISPATCHTYPE — EMERGENCY • HIGH PRIORITY • MEDIUM PRIORITY

↓

Step A — unify availability + distance into one ETA

Distance can't be compared to wait-time directly. Convert distance to travel time using the average truck speed, then add the wait, so every technician collapses to one number: minutes-until-on-site. Lower is better.

```
travel_i = (dist_i / AVG_SPEED) * 60    # miles -> minutes
eta_i = free_i + travel_i                # total minutes until tech is on site
```

↓

Step B — weighted score

Blend ETA and visit count into one score. Both must be on a 0–1 scale where higher is better, so min-max normalize each across this candidate set, then weight by w (the speed share). A pure ranking is just an endpoint: $w=1$ sorts by ETA only, $w=0$ by visits only — no separate sort needed.

```
speed_score_i = (eta_max - eta_i) / (eta_max - eta_min)    # lowest eta -> 1.0
fam_score_i = (visits_i - visits_min) / (visits_max - visits_min) # most visits -> 1.0

score_i = w * speed_score_i + (1 - w) * fam_score_i        # w = speed weight
# sort score_i descending -> rank 1 = offer target
```


Edge cases: a single candidate is rank 1 by default; if all ETAs tie the speed term is flat and familiarity breaks the tie (and vice-versa).

↓

Step C — pick w from the dispatch type

Emergency and Medium are just the two ends ($w=1$, $w=0$) of the same combined formula — so one code path serves all three.

Dispatch Type	Strategy	w (speed)	Result
Emergency	Speed-Based (locked)	1.0	Pure ETA; familiarity ignored — lowest combined time wins
High Priority	Combined	0.2 (Speed 20 / Fam 80)	Weighted blend; weight configurable per strategy
Medium Priority	Familiarity-Based	0.0	Pure visit count — most familiar tech wins

One formula, three behaviors: the dispatch type only sets w . Emergency ($w=1$) reduces to pure speed, Medium ($w=0$) to pure familiarity, High blends them — no separate ranking code per type. The resulting ordered shortlist feeds directly into step 4 (offer to the top-ranked technician) in §5.

Future factor — skill gate: when technician skill data is available ($1 \leq \text{skill } i \leq 5$) and a job carries a minimum required skill, it enters as a hard gate, not a score term. Filter it in step 1 when building the candidate list — keep technician i only if $\text{skill } i \geq \text{job.minSkill}$ — so under-skilled technicians never reach ranking. The speed/familiarity scoring is untouched; only the candidate set shrinks.

B Job Summarisation — Summarise Pipeline & Retrieval

The 12-section ticket followed end to end, plus the retrieval mechanics and measured baseline it stands on.

The Summarise Pipeline

One request followed end to end: what happens between the technician reaching the Review step and the 12-section ticket appearing.

1 Mobile app compiles the payload

The wizard answers (reason, parts, time, notes, completed forms, dropdowns decoded to labels) are packed client-side with zero extra API calls, then sent as POST `/api/v1/summarise` with the app's existing auth token.

2 .NET proxy validates & forwards

The Microservices API validates the OAuth2/Entra token, strips user credentials, adds the internal API key, and forwards server-to-server to the private FastAPI service with an extended timeout.

3 Parallel enrichment (fail-soft)

Three reads fire simultaneously against the existing APIs: dispatch details (GetAssignedDispatchesDetails), per-equipment fields (GetDispatchEquipmentFieldData), and customer procedures (GetDispatchProcedures, always fetched live, never cached). If any fetch fails, the pipeline continues without it.

4 Prompt construction

A versioned ticket standard (ticket_standard.md) forms the system instruction; the user prompt assembles dispatch context, equipment specs, completed forms, wizard answers, and procedures. Token profile ≈ 2,080 in / 1,050 out per dispatch.

5 LLM call to the client's own model

The pipeline calls qwen3-vl on the client's private gateway (measured ~6–7s per call, 30–35 tok/s; data never leaves client infrastructure). The provider abstraction in llm_client.py stays, so a different model can be swapped in by config later, but no external provider is wired up. Returns the raw ticket + procedure checks as JSON.

6 Parse, guarantee, post-process

The response is parsed and normalised against the canonical 12-section list: absent sections get an explicit MISSING placeholder, so the app always receives exactly 12 sections in standard order. Deterministic Python overrides then apply wizard-type rules (e.g. Sign-Off rebuilt from the last time entry; Refrigerant Tracking set N/A when no refrigerant was used).

7 Review, fix gaps, submit

The technician sees the ticket with per-section status badges and the procedure checklist; critical gaps must be fixed inline before submit. Submission then flows through the existing mobile-post write-back unchanged, and the accepted summary is persisted to the summaries collection.

How retrieval works

Everything the copilot retrieves is deterministic SQL over the client's real dispatch history: explainable, zero new infrastructure, and already measured against a full year of staging data (252,615 dispatches). This is what replaced the earlier vector-index plan after the 2026-07-07 architecture call.

Need	Mechanism
Same-site history (briefing)	Keyed SQL query: past visit digests for the same customer and site, most recent first.
Similar past issues (briefing)	Deterministic lookup: 2–3 recent dispatches with the same equipment type and problem code are pulled and handed to the LLM as context. No embeddings, no similarity index.
Part suggestions	SQL co-occurrence: a frequency table keyed on (equipment manufacturer type × problem code) → most-used parts, with fallback to equipment type alone, then global top parts. One SQL aggregation, fully explainable.
Compliance procedures	Always fetched live from the source system, never cached or embedded: they are compliance-critical and change.

The measured baseline (backtested on staging, 2026-07-08)

The co-occurrence method was trained on 11 months of real dispatches and tested on the final month (4,151 dispatches with installed parts, 3,100 distinct parts in play). “Hit” means at least one actually-installed part appeared in the top-k suggestion list:

Suggestions shown (k)	Hit rate
Top 1	18.6%
Top 3	31.8%
Top 5	39.9%
Top 10	52.1%

Why this matters: with 3,100 possible parts, a zero-ML lookup already puts a right part in a 5-item list 40% of the time. This is the baseline any classical-ML model, embedding index, or LLM-heavy approach must beat to justify its added cost and complexity, exactly the “capture the baseline, commit to no accuracy numbers” framing agreed on the call.

Label-quality reality (what the EDA must account for)

An audit of the last 12 months of staging data showed the structured outcome codes are far weaker than they look: root-cause and failure codes have been dead since 2021, the problem code's top value is OTHER at 38%, and the summary code is SERVICE for 88% of filled rows. The genuine predictive signal lives in the free-text complaint (DispatchReason, 230k+ distinct values) and technician notes, plus equipment identity. Any classical-ML phase therefore must include text feature engineering; training on code columns alone will underperform the baseline above.

The vector index: optional, criteria-gated

Question	Answer
Is it part of this architecture?	No. Decided 2026-07-07: it added infrastructure cost and operational surface without validated benefit.
When would we add it?	Only if evals show deterministic retrieval misses relevant history that semantic search over the free-text fields would find, and the gain beats the baseline by enough to pay for the pipeline.
How would it slot in?	Behind the existing retrieval abstraction: the briefing builder swaps its SQL lookup for a filtered similarity query; nothing else in the architecture changes. Embeddings can run on the private gateway's nomic-embed model, keeping data in client infrastructure.

C Parts Finder — Matching & Sourcing Logic

Prediction is retrieval-augmented. When a technician opens a dispatch, the app finds the most similar past dispatches from a vector database using cosine similarity, then suggests the parts those similar jobs actually

used — closer matches count for more. The index updates itself as dispatches close, so suggestions keep improving over time.

Once parts are chosen, the source step works out where each one can be collected across the technician's truck, other warehouses, and the United vendor. The detailed scoring and retrieval mechanics for the AI use cases are documented separately.

07 Data & Schema

Three data footprints, one per use case, lifted at full fidelity from each source design. A coordinates purely through ephemeral Amazon ElastiCache for Redis structures — no data model of its own. B and C each persist AI output into the AI server's own MongoDB on Azure (Azure Cosmos DB for MongoDB, vCore) — a separate datastore from the AWS SQL Server system of record, with zero coupling to the business tables (§3). C additionally maintains a semantic vector index over historical dispatches for retrieval.

A Auto-Dispatch — Amazon ElastiCache for Redis Data Structures

The three Redis structures that coordinate dispatch across the ECS Fargate worker pool.

Structure	Key	Purpose
STREAM	dispatch:stream	Durable buffer of incoming dispatch requests between the .NET Legacy API on AWS EC2 and the ECS Fargate workers. Consumed via a consumer group (XREADGROUP) — all N workers compete; each message goes to exactly one worker.
MUTEX	lock:tech:{techId}	Per-technician distributed lock. A request acquires locks on its candidate technicians before offering, in a fixed order (e.g. ascending techId) to avoid deadlock between overlapping requests. Requests with disjoint candidates run in parallel — even in the same division; requests sharing a technician serialize on that lock, so a tech is never double-offered. TTL self-releases on a crashed worker.
LIST	resp:{dispatchId}:{techId}	Per-technician reply channel. The AWS EC2 API LPUSH es the Accept/Reject; the worker BLPOP s it — instant wake, no polling. EXPIRE 60s self-cleans.

Concurrency: a worker processes up to 20 dispatches concurrently using async (redis.asyncio) — a blocked BLPOP on one dispatch never stalls the others.

Diagram note: the §2 architecture diagram still shows a per-division lock (lock:div:*) as a simplification. The implemented model locks per candidate technician as described here — diagram redraw pending.

B Job Summarisation — the AI MongoDB Collections

All AI output lives in the AI server's own MongoDB (Azure Cosmos DB for MongoDB, vCore): a datastore owned entirely by the FastAPI AI service, with no connection to the AWS SQL Server system of record and therefore zero mixing with business tables.

Indicative document shapes below; final collection and index design is agreed with the client DBA.

summaries

Field	Type / notes
_id	ObjectId · primary key

Field	Type / notes
dispatchId	int · reference to the SQL Server dispatch, carried in the request payload — not a database foreign key
sections	array · the 12 sections
procedureResults	array
status	draft / accepted
acceptedAt	date
embedding	array<double> · optional, stage 1

llm_calls

Field	Type / notes
_id	ObjectId · primary key
dispatchId	int
provider / model	gateway · qwen3-vl
tokensIn / tokensOut	int
latencyMs	int
outcome	ok / parse_fail / error

summary_edits

Field	Type / notes
_id	ObjectId · primary key
summaryId	reference → summaries._id
section	which of the 12
beforeText / afterText	the correction
editedBy / editedAt	who · when

About the embedding field: it is a placeholder for the optional future vector layer (\$6). Because MongoDB stores it as a native array of doubles, the SQL Server 2022 Web Edition constraint that previously forced a varbinary column no longer applies. It stays empty unless that layer is added. Open client items: collection and index design sign-off, and the dispatch-assignment trigger mechanism.

C Parts Finder — Data Model

Two stores cooperate. The existing SQL Server system of record (FieldJetX) owns the business data the Parts Finder reads — dispatches, the inventory catalogue, and stock by location — and the AI service only ever reads it, never writes to it. A separate vector index holds the semantic search index used for prediction, and

the AI service's own MongoDB holds the data it writes: the parts a technician commits, acceptance signals, and indexing bookkeeping.

The relational database stays the single source of truth. Nothing in the existing business tables changes, and the vector index is derived and rebuildable.

08 API Reference

The full HTTP surface across all three use cases, lifted at full fidelity from each source design — every endpoint, parameter, and response shape kept intact. Only the caller-facing compute labels are updated to match this document's Azure-first infrastructure (§2 – §4) and the service renames already established there. Nothing in the request/response bodies themselves changes.

A Auto-Dispatch — API Reference

New endpoints introduced for Auto-Dispatch. Two callers: the admin dashboard (control plane — toggle & config) and the technician mobile app (data plane — push registration & replies). All existing endpoints (dispatch ingestion, dispatch status, technician location) are unchanged and out of scope here.

Method	Path	Caller	Purpose
POST	/divisions/{id}/toggle	Dashboard → AWS EC2	Turn auto-dispatch ON/OFF for one division. Body { enabled: bool }. On enable: pull pending requests into the stream and scale the ECS Fargate pool up; on disable: drain and scale to zero (idle = \$0).
PUT	/divisions/{id}/config	Dashboard → AWS EC2	Update per-division ranking weights — distance, familiarity, priority, availability window. Body { weights: {...} }. Applied to the next assignment loop; no restart needed.
GET	/divisions/{id}/config	Dashboard → AWS EC2	Read current toggle state + weights so the admin UI can render the switch position and weight sliders. Paired read for the two writes above.
POST	/devices/register	Mobile → AWS EC2	Register/refresh a device push token. Body { techId, platform: ios android, token }. AWS EC2 registers the device with Amazon SNS and stores techId → endpointArn. Re-posted whenever the OS rotates the token, so pushes never silently die.
POST	/technicians/{id}/availability	Mobile → AWS EC2	Technician reports when they will be free. Body { availableAt }. Not tied to any dispatch — it updates the technician's general availability state, which the ranking loop reads for whichever request is being processed.
POST	/dispatches/{id}/respond	Mobile → AWS EC2	Technician accepts/rejects a specific offer. Body { accept: bool }. Deduped via SETNX, then LPUSH ed to resp:{dispatchId}:{techId}; the worker's BLPOP wakes instantly.

Authentication

No new auth mechanism is introduced. Every endpoint sits behind the platform's existing auth middleware, by caller:

- Control plane (POST /divisions/{id}/toggle, PUT /divisions/{id}/config, GET /divisions/{id}/config) — behind the admin Dashboard auth, identical to the existing Dashboard endpoints. Only authenticated admins can toggle or reconfigure a division.
- Data plane (POST /devices/register, POST /technicians/{id}/availability, POST /dispatches/{id}/respond) — behind the technician mobile-app auth, identical to the existing technician endpoints. The authenticated technician identity must match the techId /technician in the path or body.

Error contract

All endpoints return the platform's standard error envelope so a client can handle failures uniformly:

```
{ "code": "string",  
  // stable machine code, e.g. DIVISION_NOT_FOUND  
  
  "message": "string",  
  // human-readable  
  
  "requestId": "string" }  
// correlation id for tracing
```

Global status codes apply across all endpoints:

- 400 — malformed body / invalid field (e.g. weight out of range, unknown platform).
- 401 — missing/invalid credentials · 403 — authenticated but not permitted (e.g. non-admin hitting a control-plane route, technician acting for another techId).
- 404 — division / dispatch / technician not found.
- 409 — duplicate reply on /dispatches/{id}/respond (the SETNX dedupe already rejected it); the first reply stands.
- 500 — unexpected server error (correlate via requestId).

Two response types — two separate endpoints

The technician's two replies live at different paths because they have different scope:

- Availability — POST /technicians/{id}/availability. Tech-scoped, no dispatchId. A general "I'm free at ..." answer to the availability poll during the ~1 min ranking window; updates the technician's availability record that ranking reads.
- Decision — POST /dispatches/{id}/respond. Dispatch-scoped. Accept/reject one specific offer; drives the offer loop, and a reject (or timeout) moves to the next ranked technician. This is the reply the worker is blocked on (BLPOP).

B Job Summarisation — API Reference

The API surface is plain REST with Pydantic models (the earlier prototype's GraphQL layer is refactored away). The service is reachable only through the .NET proxy; there is no public endpoint.

Method	Route	Purpose	Typical latency
POST	/api/v1/summarise	Wizard payload → 12-section ticket + procedure check results	~6–7s (LLM-bound)
POST	/api/v1/analyse	Ticket → gap findings + inline remediation fields; re-run until clean	~6–7s
GET	/api/v1/suggestions	Dispatch context (equipment type + problem code) → top-k part suggestions from the co-occurrence tables; deterministic, no LLM call	<200ms
GET	/admin/dashboard	Observability dashboard: edit-rate per section, latency, cost, parse failures, read from the AI MongoDB collections. Served by the same FastAPI container; no separate web app.	<500ms
GET	/health	Liveness probe for the Container Apps health check	<50ms

Auth model: no client talks to these routes directly. The .NET proxy is the only caller, authenticated by an internal API key plus network trust (private Azure VNet, security rules). The dashboard follows the same rule: it is exposed through a .NET-proxied route with an admin-role check on the existing Entra token. This keeps a single user-auth system and zero new login surfaces.

C Technician Parts Finder — API Reference

The FastAPI AI Server exposes a deliberately small HTTP surface, reached by FJX 2.2 through the .NET Microservices API's AI proxy route — the mobile app never calls the AI layer directly, and the proxy forwards exactly the two control endpoints (/parts/predict, /parts/source).

Endpoint	Purpose
GET /health	Liveness / deploy probe (load-balancer + Container Apps health checks; not proxied to clients).
POST /parts/predict	RAG-predicted parts for a dispatch — via the AI proxy route.
POST /parts/source	All sources per part — via the AI proxy route.

Scope: everything else the flow touches stays where it already lives: dispatch list/detail and the manual-add inventory tabs / catalog search are served by the existing .NET APIs, unchanged. The AI server does not serve or duplicate them — it reads the dispatch reason and stock context from the Legacy API, and calls the United CP Gateway server-side for the supplier tier.

POST /parts/predict

```
// request
{
  "dispatchId"
  :
  "...
  }
// response → PredictedPart[]
```

```
[ {
  "inventoryId"
  :
  "C066"
  ,
  "name"
  :
  "Condenser Fan Motor 1/4 HP"
  ,
  "partNumber"
  :
  "C066"
  ,
  "confidence"
  : 88,
  // 20-95

  "selected"
  : true,
  // pre-selected when confidence ≥ 60

  "timesUsed"
  : 3
} ]
```

Returns 503 when the prediction subsystem (embedding model / vector store) is unavailable — the client falls back to manual add. 502 on an upstream failure during prediction.

POST /parts/source

```
// request

{
  "inventoryIds"
  : [
    "C066"
  ,
    "24708"
  ],
  "dispatchId"
  :
  "..."
  ,
  "priority"
  :
  "Normal"
}

// response → SourcedPart[]

[ {
  "inventoryId"
  :
  "C066"
```

```
,
"name"
:
"Condenser Fan Motor 1/4 HP"
,
"partNumber"
:
"C066"
,
"qtyNeeded"
: 1,
"sources"
: [ {
"tier"
:
"truck"
,
// truck | warehouse | vendor
"label"
:
"Your Truck — IL01 WH V1885"
,
"detail"
:
"Bin C2"
,
"distanceMi"
: 0.0,
"qtyAvailable"
: 2,
"price"
: null
// null for own stock; $ for vendor
} ]
} ]
```

Supplier integration — United CP Gateway (server-side, outbound)

For the vendor tier of /parts/source, the AI server calls the ClimatePros API Gateway for United Refrigeration from the backend — the contract below is the gateway's own shape, so the integration is a drop-in and the subscription key never leaves the server:

```
// request
{
"item"
:
"C066"
```

```
,
"coordinates"
: {
"latitude"
: 21.30,
"longitude"
: -157.85 },

"qty"
: 1,
"miles"
: 50,
"branchesIn"
: [],
"limit"
: 1 }

// response

{
"code"
: 100,
"success"
: true,
"response"
:
"OK"
,
"errors"
: [],

"inventory"
: [ {
"branchNo"
,
"address1"
,
"city"
,
"state"
,
"zipcd"
,
"phone"
,
"miles"
,
"workingHours"
,
"item"
,
"qtyAvail"
} ] }
```

Contract notes

Concern	Behavior
Auth	Bearer token from the FJX 2.2 session, validated at the .NET Microservices API before the AI proxy route forwards the call; the AI server is reachable only from inside the VNet. Supplier gateway calls use a subscription key held server-side (see §12 Non-goals & PII).
Errors	400 validation · 404 dispatch not found · 502 upstream failure · 503 prediction unavailable.
CORS	Restricted to the FJX 2.2 mobile origin / native client in production.
Idempotency	Predict and source are read-only and side-effect free; safe to retry.

09 Observability & Evals

Two different concerns share this section. Observability is operational and unified: one tracing/logging layer, introduced fresh by this project (§3 Technology Stack, §4 Infrastructure), watches all three use cases the same way. Evals are about AI output quality and only apply where a model is in the loop — Auto-Dispatch (A) is a deterministic ranking formula (§6 Core Logic) with nothing to eval; Job Summarisation (B) already has a working, production-grounded eval loop, lifted at full fidelity below; Parts Finder (C) today only validates retrieval quality on a held-out set of recent dispatches and does not yet have B's correction-based feedback loop — closing that gap is future work, not part of this architecture.

Unified Observability — OpenTelemetry Tracing + CloudWatch / Azure Monitor Logs

Every service in Stream 1 — the AWS EC2 host running both .NET APIs, the AWS ECS Fargate dispatch-worker pool (A), the Azure Container Apps FastAPI AI server (B, C), the Redis-fed worker loop, and every vector-store call — emits OpenTelemetry traces and structured logs. AWS-side services report into CloudWatch; the Azure AI stack reports into an Azure Monitor workspace, and trace context is propagated across the single AWS→Azure hop so one dispatch is one trace (§3, §4). This was an abstract, “not yet designed” placeholder in the source design; here it is the one piece of infrastructure every use case shares, so it is specified once instead of three times.

Distributed traces

One trace per request, spanning every hop it takes — .NET proxy → Container Apps → AI server → LiteLLM → model, or .NET → Redis Stream → worker → SQL Server. The cross-cloud hop to AWS SQL Server (§4) is a span like any other, so the added round-trip is visible, not hidden.

Structured logs

Every service logs to Azure Monitor in one schema. Each entry carries the same requestId issued in the error envelope (§8 API Reference), so a failed call traces from the client-visible error code back through every hop it touched.

Correlation

requestId is the join key across traces, logs, and the AI-specific tables in §7 Data & Schema (the llm_calls collection, the summary_edits collection) — so a slow trace, a log line, and an eval finding for the same request all resolve to one id.

P95 targets — open item: §4 Infrastructure flags that cross-cloud AWS SQL Server round-trips must be budgeted into per-flow P95 targets, especially the Auto-Dispatch worker's per-request SQL calls. No formal SLOs exist yet in any of the three source designs; the inputs this document already establishes are the ~1 min technician-availability window and BLPOP reply-wait timeout (A, §5 Operational Flows), the ~6–7s LLM-bound /summarise / /analyse calls (B, §8), and the <200ms / <50ms deterministic endpoints (B/C, §8). Setting

numeric P95 targets per flow, inclusive of the cross-cloud SQL leg, is follow-up work once Azure Monitor is live and has real traffic to baseline against — not something any source design specified.

B Evals — the Feedback Loop

Every AI interaction is persisted in the client's own database, so quality is measured from production reality, not vibes.

The feedback loop

Log everything → corrections become ground truth → changes must beat the eval set.

- the `llm_calls` collection records provider, model, tokens, latency, and outcome for every call: cost and reliability are queryable at any time.
- the `summary_edits` collection records every human correction to an AI section: this is labelled ground truth generated by normal usage.
- Eval set is built from those corrected rows and re-run before any prompt, ticket-standard, or provider/model change ships.
- Headline metric: edit-rate per section: which of the 12 sections the AI gets right, tracked over time and per customer.

Traceability: because prompts are built from a versioned `ticket_standard.md` and every call is logged with its model id, any ticket in production can be traced back to exactly which standard version and model produced it.

How we view it: a lightweight dashboard page (GET `/admin/dashboard`, §8 API Reference) is served by the same FastAPI container and reads straight from the AI MongoDB collections: no separate web app, no new hosting. Access is proxied through the .NET backend with an admin-role check on the existing Entra token, the same pattern as every other AI route, so the service stays fully private and no new login surface is created. Eval runs themselves are offline jobs over the `summary_edits` collection, not a hosted service.

The eval set inherits weak labels, not clean ones: the same audit that shaped the retrieval baseline (§6 Core Logic · B · Label-quality reality) found the structured outcome codes are mostly dead signal — root-cause/failure codes unused since 2021, problem code OTHER at 38%, summary code SERVICE for 88% of filled rows. The real predictive signal is free text (DispatchReason, technician notes). the `summary_edits` collection works around this because it labels the model's actual output, not the weak structured codes — but it also means the eval set is only as good as what technicians bother to correct, not an independently audited ground truth.

10 Cost Projection

One shared infra bill across A/B/C — same compute tiers, same fixed floor. A use case only shows up as a load driver (dispatch hours-on, B+C call volume), not a separate section.

Figures below are for **one environment only** (e.g. prod). Dev/Staging add cost on top — each environment duplicates this bill.

Pending re-pricing: Azure lines (Container Apps, vector store, Monitor) below use carried-over AWS rate-card estimates, not real Azure prices yet. Cosmos DB (Mongo vCore) is a real Azure quote, unverified for region/tier.

Compute Sizing

New tiers scale horizontally (more replicas), not vertically (bigger box). Baseline 0.5 vCPU / 1 GB handles 60+ concurrent dispatches per instance.

Component	Instance size	Scaling	Notes
Dispatch workers (ECS Fargate) — AWS	0.5 vCPU / 1 GB	Scale 0–1 on Auto-Dispatch toggle	60+ concurrent dispatches/instance
AI server (FastAPI) — Azure	0.5 vCPU / 1 GB	Scale 1–N on request concurrency	Shared B & C; min 1 replica, no cold-start
ElastiCache for Redis — AWS	Managed tier	Fixed; sized to queue depth	No-eviction, persistence on
Vector store — Azure	Managed, serverless/free-tier scale	3–4K embeddings — small, flat	Parts index (C); optional similar-incident index (B)
MongoDB (Cosmos DB for MongoDB vCore) — Azure	M10 tier (2 vCore / 8 GB)	Fixed tier; step up manually, not autoscale	AI stack's own store — summaries, part results, llm_calls, evals; no SQL Server coupling

Cost Formulas

AWS Fargate — dispatch workers

$\text{per_task_hour} = (\text{vCPU} \times \text{rate_vcpu}) + (\text{GB} \times \text{rate_mem})$
 $\text{monthly_cost} = N \times H \times 30 \times \text{per_task_hour}$

$\text{vCPU} = 0.5, \text{rate_vcpu} = \$0.04048/\text{hr}$
 $\text{GB} = 1, \text{rate_mem} = \$0.004445/\text{hr}$
 $\rightarrow \text{per_task_hour} = \0.024685

$N = \text{tasks (0 or 1 — scale-to-zero)}$
 $H = \text{hours ON per day}$

Fixed / near-fixed lines

ElastiCache Redis = $\$0.032/\text{hr} \times 730 \text{ hr} = \sim \$23.36/\text{mo}$ (fixed, no-eviction)
 SNS push = $500 \text{ dispatches/day} \times 5 \text{ techs} \times 30 = 75,000 \text{ pushes/mo}$
 $= 75,000 \times \$0.50/1,000,000 = \sim \$0.04/\text{mo}$
 Container Apps (AI) = $\$20 - \$40/\text{mo}$ (1-2 replicas, 0.5 vCPU / 1 GB)
 Vector store = $\sim \$5/\text{mo}$ flat (3-4K embeddings, serverless/free-tier scale)
 Azure Monitor + OTel = $\$15 - \$30/\text{mo}$
 Cosmos DB Mongo(vCore) M10 = $\$0.129/\text{hr} \times 730 \text{ hr} = \sim \$94/\text{mo}$ flat

Unified Monthly Cost by Scenario

TOTAL/MO	~\$159 LIGHT (2H/DAY)	~\$182 – \$185 TYPICAL (8H/DAY)	~\$220 – \$235 HEAVY (24x7)
Component	Light (2h/day)	Typical (8h/day)	Heavy (24x7)
Dispatch workers — Fargate	~\$1.48	~\$5.92	~\$17.77
AI server — Container Apps	~\$20	~\$30	~\$40
Redis (fixed)	~\$23.36	~\$23.36	~\$23.36
SNS push	~\$0.04	~\$0.04	~\$0.04
Vector store (flat)	~\$5	~\$5	~\$5
Monitor + OTel	~\$15	~\$22	~\$30
MongoDB — Cosmos DB vCore (flat)	~\$94	~\$94	~\$94
Variable — per B+C call	< \$1	~\$2 – \$5	~\$10 – \$25
Total/mo (est.)	~\$159	~\$182 – \$185	~\$220 – \$235

Fixed floor (Redis + vector store + Mongo + Monitor) ≈ \$137–\$152/mo regardless of traffic; only Fargate + Container Apps + variable calls scale down toward zero.

Database & Cross-Cloud Egress

SQL Server stays on AWS, unchanged — no incremental cost. Only cross-cloud line: the AWS→Azure AI request/response (B/C egress).

Component	Cloud	~ Monthly
AWS SQL Server	AWS	Existing, sunk — no incremental cost
Cross-cloud egress (AWS → Azure AI calls)	AWS egress	TBD — needs a traffic estimate; get an AWS Data Transfer OUT quote once real volume is known

11 Onboarding & Fault Tolerance

Two questions per use case: how it gets from an empty production environment to live technician traffic, and how it behaves when something it depends on fails. The rollout shape is identical for all three — only the contents of each phase differ.

Deployment follows the existing FieldJetX development team's process. Their pipelines, their staging → production promotion, their release windows, their rollback. Stream 1 adds no new deployment tooling: the dispatch worker image and the `.NET` proxy route ride the client's established pipeline on the shared EC2 host, and any client-owned code change is scheduled with their team rather than ours.

How every rollout runs

Phase	Purpose	Gate before the next phase
0 • Provision	Stand up the infrastructure the feature cannot start without — network, hosting, queues, stores, credentials.	Every dependency reachable and health-checked
1 • Seed	Load the data the logic reads on its very first request: config, schema, indexes, history.	Nothing is empty at first call — no cold start
2 • Roll out	Ship the client experience behind a flag or app-version gate, to a pilot cohort only.	Pilot cohort live; everyone else unchanged
3 • Go live	Monitoring first, then one division/branch, then expand.	Pilot clean on its own metrics before widening

Each environment is fully isolated — separate queues, databases, indexes and keys — so a staging run can never touch live technician or customer data.

A Auto-Dispatch

Onboarding

Phase	What Auto-Dispatch needs	Cleared when
0 • Provision	ElastiCache for Redis cluster up with the consumer group created (<code>XGROUP CREATE dispatch:stream</code>) — workers cannot read the stream before it exists. Worker image in ECR, ECS Fargate task definition, scale 0 ↔ N policy. Amazon SNS platform applications for APNs (Apple <code>.p8</code>) and FCM. IAM: SNS publish, ECS scale control, ElastiCache access.	Workers consume the stream, and <code>/devices/register</code> has live push credentials behind it

Phase	What Auto-Dispatch needs	Cleared when
1 • Seed per division	Ranking weights — distance, familiarity, priority, availability window — written into SQL Server. Toggle defaults OFF for every division. Escalation target (the human dispatcher queue) defined. Technician records carry division, location source and skills.	Config and an escalation target exist for every division; nothing auto-dispatches yet
2 • Roll out push	Push-enabled app build with a minimum-version gate. New technicians register at sign-up; existing ones register on first open of the updated build. The app re-registers on OS token rotation and a periodic refresh, upserting on (<code>techId</code> , <code>token</code>).	Push-adoption % climbing per division
3 • Go live	Monitoring first — push adoption, stream lag, lock-hold time, BLPOP timeout rate. Pilot one division, validate exactly-once assignment and escalation end to end, then expand division by division.	Pilot clean; manual fallback retired as adoption nears 100%

The one cohort with a gap: a technician who has not yet opened the push-enabled build has no installation id, so Amazon SNS cannot ring them. They are skipped to manual assignment — never offered a job they cannot receive — and rejoin automatically on their next token refresh. No migration job and no per-technician provisioning.

New personal data: the push token and its SNS `endpointArn` (`techId` → `endpointArn`) are the only new PII this project stores — encrypted at rest, limited to the dispatch backend, overwritten on rotation and cleaned up when dead. Technician location is read from the platform's existing system, not newly stored.

Fault Tolerance

A dispatch is handed to exactly one worker, and the queue tracks it as *handed out but not finished* until that worker marks it done.

Failure	Behaviour	Why it is safe
Worker dies mid-dispatch	The request stays flagged unfinished; after the visibility timeout another worker claims and completes it	Nothing is lost — the request is recovered without an operator
Worker is slow, not dead	It still holds the per-technician claim, so the worker attempting takeover is blocked	No double offer — the same job never reaches two technicians
Worker truly crashed	Its claim releases automatically after a short hold, and a healthy worker takes over	The claim is the arbiter: still held means stay out, released means go ahead
No technician accepts	The 30-second offer window cascades down the shortlist, then escalates to a human dispatcher	A job is never silently dropped

Net effect — exactly-once: every request is processed even if a worker dies, and no request is processed twice even if a worker is slow.

B Job Summarisation

Onboarding

Phase	What the copilot needs	Cleared when
0 • Provision	Private subnet and internal load balancer (idle timeout $\geq$ 120 s) — the AI service gets no public URL. FastAPI image in the registry, always-on at min 2 instances on Azure Container Apps (the shared service that also serves C). Network path and key for the client's private model gateway. Secrets — gateway key, internal proxy key, <code>FJX_API_TOKEN</code> — in Azure Key Vault, never in code or images. The <code>.NET</code> proxy route is client-owned code, so it is scheduled with their team.	AI service reachable privately; models answer; no secret outside the vault
1 • Seed	DBA sign-off on the read-only SQL Server account, then the AI MongoDB database and its collections (<code>summaries</code> , <code>llm_calls</code> , <code>summary_edits</code>) with indexes on Azure Cosmos DB for MongoDB. <code>ticket_standard.md</code> v1 agreed and frozen — every prompt and eval references a version. Co-occurrence tables seeded by one idempotent SQL aggregation over 12 months of dispatch/parts history. Pilot customers' compliance procedures verified current.	Schema, standard and history in place — gap checks have something to check against
2 • Roll out	Copilot build plus min-version gate for the pilot cohort. Older app versions keep today's manual review page and simply never call the AI route. Assignment trigger decided (poll vs <code>.NET</code> hook — open client item). Pilot cohort briefed: the AI drafts, they review, edit and submit.	Pilot branch live; every other branch unchanged
3 • Go live	Monitoring first — LLM latency and error rate, parse-failure rate, proxy timeout rate, edit-rate dashboard, all from <code>llm_calls</code> / <code>summary_edits</code> . Pilot one customer/branch on real dispatches, capture the edit-rate and call-back baseline, then expand branch by branch.	Baseline captured; any later prompt or model change passes the eval set first

Editing is contributing, not correcting. Technician edits feed the eval set (\$9), so the pilot cohort is briefed that they stay accountable for the ticket — the copilot drafts, they sign off.

Fault Tolerance

Failure	Behaviour	Why it is safe
An enrichment read fails	Pipeline continues without that block (fail-soft)	The ticket is built from what is available; missing context costs quality, not availability
Private gateway outage	AI features degrade to the manual experience — plain recap, briefings and suggestions skipped; submission never blocked	The gateway is the single model dependency and AI is an overlay, so losing it loses assistance, not function

Failure	Behaviour	Why it is safe
Gateway saturated	Calls queue with a timeout; throughput drops from 30–35 tok/s single-call to ~11 tok/s at 4 concurrent, and calls past the timeout fall back to the manual recap	Latency degrades gracefully; the ~10% pilot keeps concurrency low and capacity is watched in llm_calls
Malformed model output	Parser falls back to regex extraction, then MISSING placeholders	The app always receives exactly 12 sections — a bad generation cannot break the review page
A Container Apps instance dies	The load balancer routes to the remaining instance and Container Apps replaces the replica	Minimum 2 always-on instances, so no cold-start window
AI tier unreachable	Review page falls back to today's manual recap; submission never blocked	Worst case is the pre-copilot experience

System of record: AWS SQL Server stays the single source of truth. The AI tier writes only to its own MongoDB collections and never to SQL Server; job write-back rides the existing mobile-post path, so an AI failure cannot corrupt or block business data.

C Parts Finder

Onboarding

Phase	What the parts finder needs	Cleared when
0 • Provision	Azure Container Apps service on the private VNet with an internal load balancer, image built in CI. Azure-hosted vector store and index created; connectivity to the customer-hosted embedding model and LLM health-checked. Read-only path to AWS SQL Server plus connectivity to its own MongoDB, the .NET Legacy API and the United CP Gateway. MongoDB collections and indexes on Azure Cosmos DB; AI proxy route deployed on the same shared .NET route B uses.	All four sources and both models reachable
1 • Seed the index	Batch-embed historical completed dispatches on the self-hosted embedding model, idempotent upsert by dispatchNo . Validate retrieval quality on a held-out set of recent dispatches.	The index is never empty at go-live — the cold-start gap is closed from the platform's own history
2 • Roll out	Ship "Add Parts with AI" behind a feature flag, enabled per branch/division. Pilot a small technician cohort, watching acceptance rate and source mix (truck / warehouse / vendor, §6).	Flag on for the pilot cohort only
3 • Go live	Expand the flag division by division. Wire the on-completion re-embed pipeline so the index grows as dispatches close.	The self-learning loop runs without a separate retraining job

Fault Tolerance

Failure	Behaviour	Why it is safe
Models or vector store unavailable	/parts/predict returns 503; the client falls back to manual inventory add	The flow continues — prediction is an accelerator, not a gate
Supplier (United) outage or timeout	That vendor tier is dropped from the part's sources; own-stock truck and warehouse options still return	The request does not fail — a United outage cannot block a technician who only needed own stock
Model throttling or slowness	Retry with backoff and batched pacing at index time; request-time timeouts bound prediction latency	Transient throttling cannot corrupt the index
Duplicate or repeated indexing	Idempotent upsert keyed by dispatchNo	Re-running indexing never double-counts a dispatch
SQL Server slow or unavailable	Read replicas plus brief caching for hot stock lookups; sourcing surfaces last-known availability where safe	Degrades to slightly stale stock rather than no answer
Container Apps replica crash	Stateless replicas behind the load balancer; Container Apps replaces unhealthy ones (health-checked via /health)	No single replica is load-bearing

The vendor lookup is the only call in C that leaves the client's own infrastructure, which is why it is the one tier dropped independently rather than failing the whole [/parts/source](#) request.

Environments

Use case	Isolated per environment	Shared
A Auto-Dispatch	Own ElastiCache for Redis, own SQL Server database, own ECS Fargate service	Existing EC2 application host
B Job Summarisation	Own Container Apps service, own MongoDB database	Private model gateway, .NET proxy route
C Parts Finder	Own vector index, supplier gateway on a test key in staging and a production key in production	Container Apps service and AI proxy route shared with B

12 Non-goals & PII

Two boundaries, drawn once for all three use cases instead of three times: what each use case deliberately does not do, and what personal data this project touches. C's source design carries the only dedicated PII inventory of the three — lifted at full fidelity below — and A's one new piece of personal data (the device push token) was already catalogued in full where it is introduced, §11 Onboarding & Fault Tolerance.

Non-goals

A Auto-Dispatch

- Not route optimization — closest-available-responder, like a 911 dispatch. It ranks who answers next, not the most efficient multi-stop route.
- Deterministic, no LLM — a weighted-sum over stated inputs with configured (not learned) weights, so every assignment is explainable and repeatable.
- The existing manual dispatch flow is untouched — a division with the toggle OFF behaves exactly as today.
- No UI change beyond the toggle and its configuration (task count N).
- No change to the SQL Server system-of-record schema beyond dispatch status fields.

B Job Summarisation

- Never fabricates. The ticket is generated from what is actually available — a missing block degrades to a MISSING placeholder, never an invented one (§6 Core Logic, §8 API Reference).
- Never blocks Save. Every failure mode — malformed output, gateway outage, saturation — degrades to the manual recap; submission is never gated on the AI tier (§11 Fault Tolerance).
- Completeness over prose. The headline isn't prettier writing — that's table stakes — it's cross-checking the note against parts, time and photos actually logged and surfacing what's missing.

C Parts Finder

- No price ranking, no ordering. The tech decides where to pick a part up — the system shows the part, the place and the distance, and stops there.
- One supplier for the POC (United, via the existing CP API Gateway); more suppliers is future work, not this scope.
- Not replacing technician judgment — predictions are suggestions the tech can add, swap or ignore, never auto-committed.
- No automatic purchasing or PO creation from the vendor, and no warehouse/inventory management — stock levels stay owned by the system of record.

PII inventory & data handling

C PII inventory & security controls

AWS-specific terms are reconciled to their Azure equivalents per §3 / §4 (private Azure VNet in place of AWS VPC, Azure Key Vault in place of AWS Secrets Manager/SSM Parameter Store); AWS SQL Server and the self-hosted embedding/LLM layer are unchanged.

Area	Control
Client → service auth	Bearer token from the FJX 2.2 session, validated at the .NET Microservices API before the AI proxy route forwards the call; CORS restricted to the mobile client.
Network isolation	The AI layer runs in a private Azure VNet — the FastAPI AI Server is not internet-reachable; only the .NET proxy route (and the VNet paths to SQL Server, the vector store, and the customer-hosted models) can reach it.
Model hosting	Embedding and LLM inference run on client-provided, self-hosted models on customer hardware — dispatch text and parts history are never sent to a third-party AI provider.
Supplier gateway	United calls carry the Ocp-Apim-Subscription-Key header; the key is held server-side only and never reaches the mobile client (§8 API Reference · Contract notes).
Secrets management	All credentials (gateway key, DB, model endpoints) resolved from a secret store — Azure Key Vault — with rotation. No secrets in source or images.
Data store access	Least-privilege SQL Server account: read-scoped on inventory and dispatch tables with no write grant at all; every AI write goes to its own MongoDB. Network-restricted to the service subnet.
Transport	TLS on all hops (client, proxy, gateway, model endpoints, internal service calls).

- Dispatch & customer detail (name, address, phone) — served to the authenticated technician only; excluded from log bodies (§9 Observability & Evals).
- Van / warehouse locations — used transiently for distance ordering; not persisted beyond short-lived caching.
- The vector store holds no direct customer PII — only dispatch reason text and used part numbers — reducing the sensitivity of the derived index (§7 Data & Schema).

Derived and rebuildable: because the Azure-hosted vector store is derived from SQL Server rather than an independent source of truth, it can be re-seeded at any time — supporting deletion and data-retention requests handled at the SQL Server layer.

A Device push token — see §11

Auto-Dispatch's one new piece of personal data — the push token and its Amazon SNS endpointArn (techId → endpointArn) — is catalogued in full where it is introduced: encrypted at rest, access-limited to the dispatch backend, retained only while the device is active. See the Device token / installation id = PII callout in §11 Onboarding & Fault Tolerance · Phase 2. Technician location itself is not newly stored by this project — distance/ETA reads the platform's existing location system, unchanged and out of scope here.

B C The history B and C learn from

ClimatePros has 10–12 years of operational data — 200,000+ jobs in the last six months alone, and 16.3 million note records across the full span. It is messy by the client's own admission: ~40% of notes are ≤40 characters, and the single most common note in the database is literally “checked history.” That is the problem B and C are solving, so it is also the data both inherit — B's eval set (§9) and C's retrieval index (§6, §7) are both built on it.

10–12 yrs

OPERATIONAL HISTORY

200,000+

JOBS, LAST 6 MONTHS

16.3M

NOTE RECORDS, FULL
SPAN

~40%

NOTES ≤40 CHARS

Not trained on the raw dump. The corpus is de-duplicated down to unique cases first, and personally identifiable information is stripped before anything reaches a model — whether that model is B's summarisation/copilot pipeline or C's embedding index. This is a data-preparation step upstream of both §6 Core Logic pipelines, not a per-request runtime control like the table above.

Appendix A — Corrections applied during conversion

The source section fragments had drifted apart: §2a, §3 and §4 had been updated to the confirmed cloud placement while §5 and §7–§12 still carried an earlier Azure-first model. Every correction below applies one rule — **AI-specific components on Azure, everything else on AWS** — together with the AI-datastore decision (MongoDB on Azure, not an `ai.*` schema on SQL Server). No other source content was altered, and no figures, formulas or API parameters were changed.

Terminology replaced throughout

Was	Now	Sections	Occurrences
Azure Cache for Redis	Amazon ElastiCache for Redis	§7, §10, §11	10
Azure Notification Hubs	Amazon SNS	§5, §10, §11	4
Azure VM	AWS EC2	§2, §5, §7, §8, §9, §10, §11	17
Notification Hubs	Amazon SNS	§2, §5, §8, §10, §11, §12	15
<code>ai.*</code> tables	the AI MongoDB collections	§5	1
<code>ai.llm_calls</code>	the <code>llm_calls</code> collection	§5, §7, §9, §11	6
<code>ai.summaries</code>	the <code>summaries</code> collection	§5, §6, §7	4
<code>ai.summary_edits</code>	the <code>summary_edits</code> collection	§5, §7, §9, §11	7
<code>device installation id</code>	platform endpoint ARN	§5	1
<code>deviceInstallationId</code>	<code>endpointArn</code>	§8, §11, §12	6

Statements rewritten (each corrected a claim that contradicted the cloud rule or the MongoDB decision)

Correction	Section
intro — Azure relabelling premise removed; A prices as AWS	§10
callout — re-pricing scoped to the Azure AI lines	§10
A — variable cost is the Fargate pool	§10
A — rate provenance now matches the services bought	§10
heading — Fargate formula is the live formula	§10
Fargate task rate — flagged sizing conflict with §4	§10
code — ElastiCache line and real instance class	§10
code — SNS push line	§10
scenario table header — AWS service names	§10
table footnote — provenance and cache instance class	§10

Correction	Section
scale-to-zero note — Fargate	§10
scale-to-zero note — ElastiCache floor	§10
vector store line — flagged for Azure re-pricing	§10
formula legend — ECS Fargate tasks	§10
egress heading & intro — the seam is the AI request, not the database	§10
egress table — rows rescoped to the AI-request hop	§10
egress note — A stays in AWS; only B/C requests cross	§10
A Phase 0 — ECS Fargate + ECR, sizing conflict flagged	§11
A Phase 0 — IAM instead of Azure RBAC	§11
A environments — per-env Fargate service	§11
B Phase 1 — MongoDB collections replace the SQL ai.* schema	§11
B environments — per-env MongoDB database	§11
B fault tolerance — AI writes are MongoDB-only	§11
C Phase 0 — read-only SQL access, writes to MongoDB	§11
C Phase 0 — MongoDB provisioning replaces SQL migration	§11
data-store access — SQL account is read-only	§12
2b Use Case A caption — cloud placement corrected	§2
2b B&C caption — stale "no AWS managed service" claim removed	§2
AI datastore — ai.* on SQL Server → MongoDB on Azure	§2
2b A summary line — ECS Fargate worker	§2
Database row — AI output goes to MongoDB, not a SQL ai.* schema	§3
A step 1 — dispatch workers are ECS Fargate, not Container Apps	§5
push round-trip step 1 — SNS Publish, ECS Fargate worker	§5
push round-trip step 2 — SNS platform application wording	§5
intro — ECS Fargate task walkthroughs	§5
briefing builder — cached in MongoDB	§5
intro — AI output store is MongoDB, not an ai.* SQL schema	§7
A heading body — Container Apps pool → ECS Fargate pool	§7
STREAM row — producer/consumer are EC2 and ECS Fargate	§7
B — relational ai.* schema rewritten as MongoDB collections	§7
C intro — AI writes go to MongoDB, not a SQL ai.* schema	§7
C — ai.* schema section rewritten as MongoDB + learning-loop trigger stated	§7

Correction	Section
/admin/dashboard — reads MongoDB collections	§8
/divisions/{id}/toggle — scales the ECS Fargate pool	§8
heading — both clouds' log sinks named	§9
scope sentence — A is not an AI tier; split CloudWatch / Azure Monitor	§9
dashboard — reads MongoDB collections	§9

Appendix B — Open items

Carried from the source sections plus the two conflicts surfaced by this conversion. Each needs a ClimatePros owner before this document can move from draft to agreed.

#	Open item	Where	Why it matters
1	Dispatch-worker task size: §4 Compute sizing specifies 0.5 vCPU / 1 GB; §10 Cost Projection and §11 Phase 0 both assume 1 vCPU / 2 GB.	§4, §10, §11	The cost formula and the ECS task definition are built from this number; the two figures give different monthly totals.
2	Azure re-pricing for the AI lines — Container Apps, the chosen vector store, and Azure Monitor.	§10	Those figures are still AWS-sourced rates. Use Case A's AWS figures now stand as-is.
3	Vector store selection: §4 lists Azure AI Search / Pinecone as candidates; §3 marks the store experimental; §6 B states the index is out of scope pending evals, while §6 C actively searches it.	§3, §4, §6	C needs a store at go-live; B only if evals justify it. The selection and the C-only scope should be stated explicitly.
4	Learning-loop trigger for C: the AI server has no SQL Server connection, so dispatch completion must be pushed to it by the .NET side rather than polled.	§6 C, §7 C	Without a confirmed trigger the self-learning index never updates.
5	MongoDB collection and index design sign-off with the client DBA (replaces the earlier ai.* DDL approval).	§7 B	Determines query performance for briefing retrieval and the eval dashboards.
6	Dispatch-assignment trigger mechanism for the B briefing builder.	§5 B, §7 B	The briefing must start building at assignment time to be warm when the technician opens the job.
7	Per-division lock model: the §2 diagram still shows a per-division lock (lock:div:*) ; the implemented model locks per candidate technician. Diagram redraw pending.	§2, §7 A	The diagram and the implementation disagree on concurrency behaviour.
8	Cross-cloud network path: site-to-site VPN or VPC-to-VNet peering is the target state; the interim fallback is a public HTTPS endpoint with IP allow-listing / mTLS.	§4	Security review and the P95 latency budget in §9 both depend on which one ships.
9	Push token / endpoint ARN is the one new piece of personal data this project stores.	§11, §12	Needs to be reflected in the PII inventory and retention policy.
10	CLAUDE.md in the source repository still describes the older Azure-first model ("Azure except the AWS SQL Server database") and names Azure Notification Hubs and Azure Cache for Redis.	repository	Future edits made from that guidance would reintroduce the mismatches corrected here.